**НИТУ «МИСИС»**

---

**Кафедра инженерной кибернетики**

# ОТЧЕТ

ПО

# ЛАБОРАТОРНОЙ РАБОТЕ №8.2

*«Определение параметров видеокарты с поддержкой технологии CUDA в среде Microsoft Visual Studio»*

учебная дисциплина «Параллельные вычисления»

**Студент:** Голубев И. М., БПМ-22-ПО-1

**Преподаватель:** Ватутин Э. И.

**Оценка:**

**Дата защиты:** 27.04.2026

**2026 г.**

## Введение и постановка задачи

Основная цель работы:

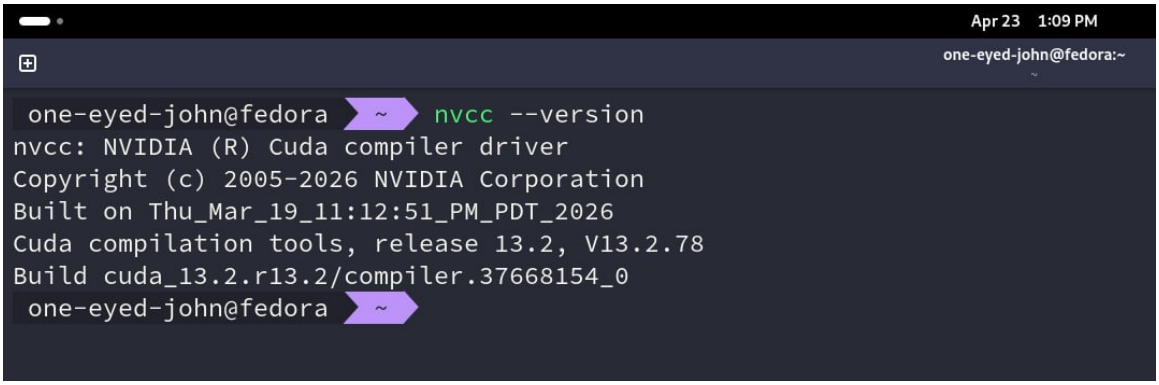
Определить число видеокарт с поддержкой технологии CUDA и следующие параметры видеокарты:

- наименование;
- общий объем графической памяти;
- объем памяти констант;
- объем разделяемой памяти в пределах блока;
- число регистров в пределах блока;
- размер WARP’а;
- максимально допустимое число потоков в блоке;
- версию вычислительных возможностей;
- число потоковых мультипроцессоров;
- тактовую частоту ядра;
- частоту памяти видеокарты;
- объем кэша второго уровня;
- ширину шины памяти видеокарты;
- максимальную размерность при конфигурации потоков в блоке и блоков в сетке.

## Выбранные средства для разработки

Для выполнения лабораторной работы использовались следующие программные средства и технологии:

- Язык программирования: C++17
- Среда разработки: Zed
- Операционная система: Fedora Linux
- Процессор: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
- Размер RAM: 15Gi
- Компиляция выполняется средствами утилиты `nvcc`




## Листинг кода программы

Код доступен в удалённом Github репозитории по ссылке:

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab08/lab08_2.cpp

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab08/lab08_2.cu

Файл lab08_2.cpp

```
extern "C" void print_cuda_device_info();

int main() {
    print_cuda_device_info();
    return 0;
}
```

Файл lab08_2.cu

```
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
```

```
static void checkCuda(cudaError_t e) {
    if (e != cudaSuccess) {
        std::printf("CUDA error: %s\n", cudaGetErrorString(e));
        std::exit(1);
    }
}

static int getAttr(int dev, cudaDeviceAttr attr) {
    int v = 0;
    checkCuda(cudaDeviceGetAttribute(&v, attr, dev));
    return v;
}

extern "C" void print_cuda_device_info() {
    int count = 0;
    checkCuda(cudaGetDeviceCount(&count));
    std::printf("CUDA devices found: %d\n\n", count);
    if (count == 0) {
        return;
    }

    for (int dev = 0; dev < count; dev++) {
        cudaDeviceProp p{};
        checkCuda(cudaGetDeviceProperties(&p, dev));

        std::printf("Device %d\n", dev);
        std::printf("Name: %s\n", p.name);

        std::printf("Total global memory: %.2f MiB\n",
                    (double)p.totalGlobalMem / (1024.0 * 1024.0));
        std::printf("Constant memory: %.2f KiB\n",
                    (double)p.totalConstMem / 1024.0);
        std::printf("Shared memory per block: %.2f KiB\n",
                    (double)p.sharedMemPerBlock / 1024.0);

        std::printf("Registers per block: %d\n", p.regsPerBlock);
        std::printf("Warp size: %d\n", p.warpSize);
        std::printf("Max threads per block: %d\n", p.maxThreadsPerBlock);

        std::printf("Compute capability: %d.%d\n", p.major, p.minor);
        std::printf("Multiprocessor count (SMs): %d\n", p.multiProcessorCount);

        int coreClockKHz = getAttr(dev, cudaDevAttrClockRate);
        int memClockKHz  = getAttr(dev, cudaDevAttrMemoryClockRate);
        std::printf("Core clock rate: %.2f MHz\n", coreClockKHz / 1000.0);
        std::printf("Memory clock rate: %.2f MHz\n", memClockKHz / 1000.0);

        std::printf("L2 cache size: %.2f KiB\n", (double)p.l2CacheSize / 1024.0);

        std::printf("Memory bus width: %d bits\n", p.memoryBusWidth);

        std::printf("Max threads dim (block): [%d, %d, %d]\n",
                    p.maxThreadsDim[0], p.maxThreadsDim[1], p.maxThreadsDim[2]);
        std::printf("Max grid size: [%d, %d, %d]\n",
                    p.maxGridSize[0], p.maxGridSize[1], p.maxGridSize[2]);
    }
}
```

## Результаты

Сборка и выполнение программы (вторая часть задания):

```
one-eyed-john@fedora  ~/Downloads/parallel-comp/lab08  main  nvcc lab08_2.cu lab08_2.cpp -o build/lab08_2

one-eyed-john@fedora  ~/Downloads/parallel-comp/lab08  main  ./build/lab08_2
CUDA devices found: 1

Device 0
Name: NVIDIA GeForce GTX 1650
Total global memory: 3715.69 MiB
Constant memory: 64.00 KiB
Shared memory per block: 48.00 KiB
Registers per block: 65536
Warp size: 32
Max threads per block: 1024
Compute capability: 7.5
Multiprocessor count (SMs): 14
Core clock rate: 1515.00 MHz
Memory clock rate: 6001.00 MHz
L2 cache size: 1024.00 KiB
Memory bus width: 128 bits
Max threads dim (block): [1024, 1024, 64]
Max grid size: [2147483647, 65535, 65535]
one-eyed-john@fedora  ~/Downloads/parallel-comp/lab08  main
```

## Выводы

В ходе выполнения задания была разработана программа на CUDA Runtime API для определения числа CUDA-совместимых устройств и получения основных характеристик видеокарты.

В системе обнаружено 1 CUDA-устройство: **NVIDIA GeForce GTX 1650** с вычислительными возможностями Compute Capability 7.5.

Полученные параметры видеокарты: общий объём глобальной памяти — **3716 МБ**, объём константной памяти — **64 КБ**, объём разделяемой памяти на блок — **48 КБ**, число регистров на блок — **65536**, размер warp — **32**, максимально допустимое число потоков в блоке — **1024**.

Число потоковых мультипроцессоров — **14**. Тактовая частота ядра составила **1515 МГц**, частота памяти — **6001 МГц**. Объём кэша L2 — **1024 КБ**, ширина шины памяти — **128 бит**.

Предельные размеры конфигурации: максимальная размерность блока потоков **[1024, 1024, 64]**, максимальная размерность сетки блоков **[2147483647, 65535, 65535]**.

Таким образом, видеокарта корректно определяется CUDA-средой, поддерживает требуемые ограничения по конфигурации запуска и обладает характеристиками, соответствующими модели GTX 1650, что позволяет использовать её для выполнения и отладки CUDA-программ.